

Memorial Técnico

Flowkly — Portal de Solicitações Internas

Documento técnico detalhando objetivo, arquitetura, modelagem de dados, decisões de implementação, regras de negócio e critérios de avaliação do projeto.

Stack: PHP 8.3 · Laravel 13 · Inertia.js 3 · Vue 3 · TypeScript · Tailwind CSS v4 · SQLite · Pest 4

Versão do documento: 1.0

Data de geração: outubro de 2026

1. Introdução e objetivo

O Flowkly é um portal interno para abertura e acompanhamento de solicitações de suporte de TI (hardware, software, rede, acesso e outras categorias). O sistema foi desenhado em torno de dois perfis de acesso distintos — **Usuário**, que abre e acompanha suas próprias solicitações, e **TI**, que recebe, triagea e resolve as solicitações de toda a organização — com um fluxo de status único (Aberto → Em Atendimento → Concluído) gerenciado pela equipe de TI em um quadro Kanban.

Este documento descreve o objetivo do projeto, a arquitetura adotada, as principais decisões técnicas e trade-offs, a modelagem de dados, as regras de negócio implementadas e um roteiro de avaliação, de forma a permitir que qualquer pessoa avalie o projeto sem precisar inspecionar o código-fonte linha a linha antes de entender o "porquê" de cada escolha.

1.1 Problema resolvido

Equipes internas costumam reportar problemas de TI por canais informais (e-mail, chat, verbalmente), o que dificulta o rastreamento: não há um código de solicitação, não há histórico de quem mudou o quê e quando, e não há visão consolidada do volume de chamados por categoria ou status. O Flowkly resolve isso centralizando a abertura, o acompanhamento e a resolução de solicitações em um único portal, com um registro auditável de todo o ciclo de vida de cada chamado.

1.2 Público-alvo e perfis

Perfil	Pode fazer
Usuário	Criar, listar, filtrar e visualizar detalhes das próprias solicitações; editar ou excluir uma solicitação enquanto ela estiver "Aberta"; trocar a própria senha.
TI	Tudo o que o Usuário faz (TI também abre solicitações, sempre destinadas a outra pessoa de TI) + visualizar e gerenciar o Kanban com todas as solicitações do sistema, alterar o status de qualquer solicitação, editar/excluir qualquer solicitação a qualquer momento, aprovar ou recusar pedidos de acesso ao portal, gerar uma nova senha para qualquer conta, e visualizar o dashboard com métricas.

2. Escopo e requisitos atendidos

A tabela abaixo mapeia cada requisito solicitado para a sua implementação concreta no código, incluindo as decisões tomadas quando o requisito admitia mais de uma interpretação razoável.

Requisito	Status	Onde / observação
Autenticação — login/sessão/logout	Atendido	LoginController, sessão padrão do Laravel (cookie + CSRF).
Dois perfis (Usuário / TI) com permissões distintas	Atendido	Enum UserRole + middleware EnsureUserHasRole + TicketPolicy; menu lateral (LateralBar.vue) filtrado por perfil.
Criar solicitação (título, descrição, categoria, data e solicitante automáticos, status inicial Aberto)	Atendido	TicketController::store; Requests/Create.vue.

Requisito	Status	Onde / observação
Editar/excluir enquanto Aberto	Atendido	TicketPolicy::update/delete — dono só pode enquanto status = aberta; TI sempre pode. Botões condicionais no modal de detalhes (Requests/Index.vue e Kanban.vue).
Listagem com código, título, categoria, solicitante, data, status	Atendido	Tabela em Requests/Index.vue (visão do usuário) e cartões com os mesmos campos no Kanban (visão da TI).
Filtros por período, categoria, status e título	Atendido	Requests/Index.vue (filtro no backend, via query string) e Kanban.vue (filtro no frontend sobre os dados já carregados).
Fluxo Aberto → Em Atendimento → Concluído	Atendido	Enum TicketStatus; transição exclusiva da TI (TicketController::updateStatus), com histórico registrado.
Kanban para a TI visualizar/gerenciar por etapa	Atendido	Kanban.vue — 3 colunas, modal de detalhes, edição, troca de status e exclusão.
Dashboard da TI (total, abertas, em atendimento, concluídas)	Atendido — com uma decisão de escopo	DashboardController calcula as métricas das solicitações vinculadas (assigned_to) à pessoa de TI logada, não do sistema inteiro — ver nota abaixo.
Tela de detalhes com informações completas	Atendido	Modal "Detalhes da Solicitação" (categoria, solicitante, data de abertura, descrição, anexos, histórico de status), reaproveitado em Requests/Index.vue e Kanban.vue.
Backend/API + persistência SQL	Atendido	Laravel + SQLite (ver §3 e §4 sobre a escolha de Inertia.js no lugar de uma API REST separada).
Regras de negócio e validações	Atendido	Ver §7 e §8.
Tratamento de erros	Atendido	Validação via Form Requests/validate(); exceptions padrão do Laravel (403/404/419/422/500); mensagens de erro específicas nas regras de negócio (ex.: e-mail duplicado, solicitação já revisada).
Frontend consumindo a API	Atendido (via Inertia.js)	Ver nota em §4.1 sobre a arquitetura Inertia (controllers fazem o papel da API; Vue consome via requisições Inertia, não fetch/axios manual).
Documentação, deploy, SQL + dicionário, README, memorial técnico, credenciais/evidências	Atendido (gerado nesta rodada)	README.md, docs/database/schema.sql, docs/database/dicionario-dados.md, docs/credenciais-avaliacao.md e este memorial técnico.

2.1 Nota sobre o escopo do Dashboard

O pedido original especificava apenas "total de solicitações, abertas, em atendimento e concluídas", sem detalhar se as métricas deveriam ser globais (todas as solicitações do sistema) ou pessoais (apenas as vinculadas à pessoa de TI logada). Durante o desenvolvimento, a especificação de produto evoluiu explicitamente para o segundo formato ("Olá, {nome}! Aqui está o resumo das *suas* solicitações"), então o Dashboard foi implementado com esse escopo — cada pessoa de TI vê os números relativos às solicitações que lhe foram atribuídas. A visão *global* de todas as solicitações do sistema, independente de quem for o responsável, já existe e está coberta pelo Kanban.

3. Arquitetura da solução

3.1 Visão geral

O Flowkly é uma aplicação monolítica Laravel que usa **Inertia.js** como camada de ligação entre o backend e um frontend Vue 3 + TypeScript. Não existe uma Single-Page Application separada consumindo uma API REST via fetch/axios — em vez disso, cada rota do Laravel devolve um componente Vue ("página") já com os dados necessários via `Inertia::render()`, e a navegação entre páginas acontece via requisições assíncronas que trocam apenas JSON (sem recarregar a página).

Esse modelo elimina a duplicação comum entre uma API JSON tradicional e um cliente SPA — não há necessidade de uma camada de serialização redundante (tipo API Resources) nem de dois conjuntos de rotas (API + web). Os `Controllers` cumprem o mesmo papel que cumpririam numa API REST (validam entrada, aplicam regras de negócio, consultam/persistem dados) — a diferença é apenas o formato da resposta, adaptado pelo próprio Inertia.

3.2 Fluxo de uma requisição

1. O navegador faz uma requisição (navegação de link, ou envio de formulário) → 2. o middleware de autenticação/autorização (`auth, role: ...`) verifica sessão e perfil → 3. o Controller valida a entrada, aplica a Policy quando aplicável, executa a regra de negócio e monta os dados → 4. `Inertia::render()` devolve o componente Vue + as props → 5. o Vue renderiza a tela; toda interação subsequente (filtro, abrir modal, enviar formulário) dispara uma nova requisição Inertia, que repete o ciclo sem recarregar o HTML inteiro.

3.3 Estrutura de pastas (resumo)

```
app/Enums · app/Http/Controllers · app/Http/Middleware · app/Mail · app/Models ·  
app/Policies  
database/migrations · database/seeder · database/factories  
resources/js/Components · resources/js/pages · resources/js/routes (gerado) ·  
resources/js/types  
routes/web.php · routes/Auth/login.php · routes/app.php  
tests/Feature
```

3.4 Rotas tipadas com Laravel Wayfinder

O projeto usa o pacote oficial **Laravel Wayfinder**, que gera automaticamente, a partir das rotas do Laravel, funções TypeScript tipadas em `resources/js/routes/` (ex.: `tickets.update(ticketId)` devolve `{ url, method }`). Isso elimina URLs "mágicas" espalhadas pelo frontend — qualquer alteração de rota no backend é refletida nos tipos do frontend automaticamente, com erro de compilação caso o frontend use uma rota que não existe mais.

4. Tecnologias utilizadas e justificativas

Tecnologia	Por que foi escolhida
PHP 8.3 + Laravel 13	Framework maduro, convenções fortes (Eloquent, migrations, validação, autenticação), produtividade alta para o escopo do teste.
Inertia.js 3	Permite construir a experiência de uma SPA (navegação sem reload, estado de formulário local) sem o custo de manter uma API REST versionada e documentada separadamente — ver §3.1.
Vue 3 (Composition API, <script setup>) + TypeScript	Componentes pequenos e tipados; os tipos de domínio (resources/js/types) espelham os Enums/Models do backend.
Tailwind CSS v4	Estilização utilitária consistente, com suporte nativo a dark mode (prefers-color-scheme) usado em todas as telas.
SQLite	Zero configuração para rodar localmente/avaliar o projeto (não exige subir um serviço de banco separado); schema é padrão (chaves estrangeiras, tipos simples) e migra para MySQL/PostgreSQL sem mudanças estruturais — ver docs/database/.
Pest 4 (sobre PHPUnit)	Sintaxe mais enxuta para os testes de feature; 35 testes cobrindo autenticação, autorização por perfil, CRUD de solicitações e suas regras de negócio.
Laravel Mail (Markdown mailables)	E-mails transacionais (nova solicitação atribuída, acesso aprovado) usando os componentes de e-mail nativos do Laravel, sem dependência externa.

5. Modelagem de dados

O schema completo, comentado, está em `docs/database/schema.sql`, e o dicionário de dados completo (toda coluna, tipo, nulidade e descrição) está em `docs/database/dicionario-dados.md`. Esta seção resume as cinco tabelas de domínio e seus relacionamentos.

Tabela	Descrição
users	Contas de acesso (perfil Usuário ou TI).
access_requests	Pedidos de acesso ao portal, antes de virarem uma conta.
tickets	As solicitações abertas pelos usuários — tabela central do domínio.
ticket_attachments	Imagens anexadas a uma solicitação (até 5, no momento da criação).
ticket_status_histories	Auditoria de cada mudança de status de uma solicitação.

```
users (1) ---< tickets.user_id "solicitante"
users (1) ---< tickets.assigned_to "responsável de TI"
tickets (1) ---< ticket_attachments.ticket_id
tickets (1) ---< ticket_status_histories.ticket_id
users (1) ---< ticket_status_histories.changed_by
users (1) ---< access_requests.reviewed_by
```

Diagrama de relacionamento (notação textual). Detalhamento coluna a coluna no dicionário de dados.

5.1 Enums do domínio

Enum	Valores	Coluna(s)
UserRole	user, ti	users.role
TicketType	hardware, software, rede, acesso, outro	tickets.type
TicketStatus	aberta, em_andamento, concluida	tickets.status; ticket_status_histories.from_status/to_status
AccessRequestStatus	pending, accepted, declined	access_requests.status

Os enums são implementados como *backed enums* nativos do PHP (`App\Enums\*`), com cast automático nos Models (ex.: `'status' => TicketStatus::class`) — o valor em banco é sempre a string (VARCHAR), mas o código PHP manipula instâncias de enum tipadas, o que elimina comparações por string solta e erros de digitação.

6. Perfis de acesso e autorização

6.1 Login segmentado por perfil

A tela de login oferece dois portais ("Usuário" e "TI"); cada um leva a um formulário de e-mail/senha que informa, junto com as credenciais, qual portal foi escolhido. O backend (`LoginController::login`) autentica a credencial e, em seguida, confirma que o perfil da conta corresponde ao portal escolhido — caso contrário, a sessão é desfeita e o login é recusado. Isso evita que alguém acidentalmente "entre como TI" apenas por ter escolhido o portal errado, e mantém os dois fluxos de entrada visualmente distintos.

6.2 Middleware de rota (EnsureUserHasRole)

Cada grupo de rotas declara explicitamente quais perfis podem acessá-lo (`role:user`, `role:ti` ou `role:user,ti` para ambos). Uma tentativa de acesso fora do perfil autorizado resulta em **403 Forbidden** — testado explicitamente em `tests/Feature/Auth/LoginTest.php` e `tests/Feature/TicketTest.php`.

6.3 TicketPolicy — autorização por registro

O middleware de rota resolve "quem pode acessar esta tela", mas a regra "o usuário só pode editar a *própria* solicitação, e só enquanto ela estiver Aberta" é uma autorização por *registro*, não por rota — por isso as rotas de edição/exclusão (`PUT/DELETE /tickets/{ticket}`) ficam abertas a ambos os perfis, e quem decide se a ação é permitida é a `TicketPolicy`:

```
update(user, ticket):
    se user.role == TI → permitido
    senão → permitido apenas se ticket.user_id == user.id E ticket.status == aberta
delete(user, ticket) → mesma regra de update
changeStatus(user) → permitido apenas se user.role == TI
```

Essa separação (middleware para "quem pode entrar na rota", Policy para "quem pode agir sobre este registro específico") é o padrão recomendado pelo próprio Laravel e evita duplicar a regra de negócio em múltiplos controllers.

7. Funcionalidades por tela

Tela	Perfil	O que faz
Login (/login)	Ambos	Seleção de portal (Usuário/TI) + formulário de acesso + botão "Solicitar acesso" (abre modal).
Nova solicitação (/requests/create)	Ambos	Título, descrição, categoria, até 5 imagens, seleção obrigatória da pessoa de TI responsável.
Minhas solicitações (/requests)	Usuário	Tabela com código, título, categoria, solicitante, data, status; filtros por título, categoria, status e período; modal de detalhes com edição/exclusão condicionais.
Kanban (/kanban)	TI	3 colunas por status; cartão com número, título, categoria e solicitante; filtros por título, categoria e período; modal de detalhes com edição, troca de status, histórico e exclusão.
Dashboard (/dashboard)	TI	4 indicadores (total, abertas, em atendimento, concluídas), tabela das solicitações recentes vinculadas à pessoa logada, gráfico de distribuição por categoria.
Solicitações de acesso (/solicitacoes)	TI	Lista de pedidos de acesso com nome, e-mail e motivo; aceitar (cria a conta + envia senha por e-mail) ou recusar.
Usuários (/usuarios)	TI	Lista de todas as contas (nome, e-mail, perfil); ação "gerar nova senha", exibida em um modal para repasse manual.
Configurações (/settings)	Ambos	Troca da própria senha (exige a senha atual).

8. Regras de negócio e validações

- Uma solicitação sempre nasce com status "Aberta", solicitante e data preenchidos automaticamente pelo backend (nunca recebidos do cliente).
- A pessoa responsável ("assigned_to") precisa obrigatoriamente ter perfil TI — validado tanto na regra de validação quanto numa checagem extra de segurança no Controller.
- Enquanto "Aberta", apenas o solicitante pode editar/excluir a própria solicitação; a partir de "Em Atendimento", só a TI pode agir sobre ela (ver §6.3).
- A troca de status é exclusiva da TI e sempre grava uma linha em ticket_status_histories (de/para, quem, quando) — inclusive a criação da solicitação grava a primeira linha do histórico.
- Um pedido de acesso (access_requests) não pode ser duplicado: o sistema recusa um novo pedido se já existir uma conta com aquele e-mail, ou se já existir um pedido "pending" para o mesmo e-mail.
- Um pedido de acesso só pode ser aceito/recusado uma vez — tentar decidir novamente sobre um pedido já revisado é rejeitado silenciosamente com mensagem de erro.
- Imagens anexadas são limitadas a 5 por solicitação, 5MB cada, e validadas como arquivo de imagem (mime/extension) antes de serem armazenadas.
- A troca de senha em Configurações exige a senha atual correta antes de aceitar a nova senha (confirmada por digitação dupla).

9. Segurança

- Senhas sempre em hash (bcrypt) — nunca persistidas nem logadas em texto plano; o cast "hashed" do Eloquent garante o hash automático em qualquer atribuição.
- Proteção CSRF nativa do Laravel em todas as rotas de escrita (formulários Inertia enviam o token automaticamente).
- Autorização em duas camadas: middleware de rota por perfil + Policy por registro (ver §6).
- Senhas temporárias geradas para novas contas (aprovação de acesso) e para o "gerar nova senha" usam `Str::password(12)` do Laravel (aleatória, com letras, números e símbolos).
- Mensagens de erro de login não revelam se o e-mail existe ou não ("E-mail ou senha inválidos" genérico).
- Validação de entrada em toda rota de escrita via `Illuminate\Http\Request::validate()`, com mensagens de erro por campo devolvidas ao formulário correspondente.

10. Testes automatizados

O projeto usa Pest sobre PHPUnit, com 35 testes de feature cobrindo os fluxos descritos neste documento — login por portal, controle de acesso por perfil, CRUD de solicitações (incluindo a regra de editar/excluir somente enquanto "Aberta"), filtros da listagem, Kanban, dashboard, pedidos de acesso (incluindo o envio de e-mail, verificado com `Mail::fake()`) e gerenciamento de usuários.

```
php artisan test
```

Os testes usam `RefreshDatabase` (banco limpo a cada teste) e `factories (database/factories)` para gerar dados de teste realistas sem depender de fixtures manuais.

11. Como executar o projeto

Instruções completas (incluindo deploy em produção) estão no `README.md` na raiz do repositório. Resumo rápido para rodar localmente:

```
composer install && npm install
cp .env.example .env && php artisan key:generate
php artisan migrate --seed
npm run dev # terminal 1
php artisan serve # terminal 2
```

As credenciais de teste (criadas pelo seeder) e um roteiro de avaliação passo a passo estão em `docs/credenciais-avaliacao.md`.

12. Mapa de rotas

Método	Rota	Descrição
GET	/login	Tela de seleção de portal
GET	/login/usuario, /login/ti	Formulário de login por portal
POST	/login	Autenticação
POST	/logout	Encerra a sessão
POST	/solicitar-acesso	Cria um pedido de acesso
GET	/requests	Minhas solicitações (Usuário)
GET	/requests/create	Formulário de nova solicitação (Usuário/TI)
POST	/requests	Cria uma solicitação
PUT	/tickets/{ticket}	Edita uma solicitação (Policy)
DELETE	/tickets/{ticket}	Exclui uma solicitação (Policy)
PATCH	/tickets/{ticket}/status	Altera o status (TI)
GET	/kanban	Quadro Kanban (TI)
GET	/dashboard	Dashboard (TI)
GET	/solicitacoes	Lista de pedidos de acesso (TI)
POST	/solicitacoes/{id}/aceitar	Aprova um pedido de acesso (TI)
POST	/solicitacoes/{id}/recusar	Recusa um pedido de acesso (TI)
GET	/usuarios	Lista de contas (TI)
POST	/usuarios/{user}/redefinir-senha	Gera nova senha para a conta (TI)
GET	/settings	Configurações (ambos)
POST	/settings/password	Troca a própria senha (ambos)

13. Decisões técnicas e trade-offs

Inertia.js em vez de API REST + SPA separada

Reduz a superfície de código (um único conjunto de rotas/controllers serve tanto a navegação quanto os dados) às custas de um acoplamento maior entre backend e frontend — trade-off aceitável para uma aplicação monolítica como esta, mas que tornaria mais difícil, por exemplo, servir um app mobile nativo com a mesma API sem adaptação.

Autorização por Policy em vez de checagem manual nos Controllers

Centraliza a regra "dono pode editar/excluir enquanto Aberto" em um único lugar (TicketPolicy), testável isoladamente e reutilizável em qualquer Controller/Request futuro que precise da mesma checagem.

SQLite em desenvolvimento

Elimina a necessidade de subir um serviço de banco externo só para avaliar o projeto — qualquer pessoa com PHP instalado roda o projeto localmente em minutos. O schema não usa nenhum recurso específico do SQLite (sem JSON1, sem FTS), então a migração para MySQL/PostgreSQL em produção é direta.

E-mails com driver "log" por padrão

Em desenvolvimento/avaliação, e-mails são apenas gravados em storage/logs/laravel.log em vez de enviados de verdade — evita a necessidade de configurar um provedor SMTP só para rodar o projeto. Basta trocar MAIL_MAILER no .env para um driver real (smtp, ses, postmark, etc.) em produção; nenhum código muda.

Anexos armazenados em disco local (storage/app/public)

Suficiente para o volume do teste; para produção com múltiplas instâncias, o disco "public" do Laravel é trocável por S3 (ou equivalente) apenas alterando a configuração em config/filesystems.php — o código da aplicação não referencia o disco diretamente.

Histórico de status como tabela separada, não como coluna JSON

Permite consultas, índices e relações (quem mudou) de forma nativa no Eloquent, e deixa a tabela tickets enxuta — o histórico cresce independentemente do registro principal.

14. Possíveis evoluções futuras

- Comentários/respostas dentro de uma solicitação (conversa entre solicitante e TI), hoje não previstos no escopo.
- Notificação por e-mail também a cada mudança de status (hoje o e-mail só é disparado na criação da solicitação e na aprovação de acesso).
- SLA / prazo por categoria, com alerta visual no Kanban para solicitações atrasadas.
- Exportação da listagem filtrada (CSV/Excel) para relatórios.
- Paginação na listagem e no Kanban quando o volume de solicitações crescer (hoje as consultas carregam o conjunto completo, adequado ao volume de um piloto/teste).
- Internacionalização (hoje todo o texto da interface está em português, fixo).

15. Conclusão

O Flowkly cobre a totalidade dos requisitos funcionais solicitados — autenticação com dois perfis, CRUD de solicitações com regra de edição condicionada ao status, listagem filtrável, fluxo Kanban com histórico auditável, dashboard, tela de detalhes, e-mail transacional e um fluxo completo de autoatendimento para pedido de acesso — construído sobre uma arquitetura monolítica Laravel + Inertia + Vue deliberadamente simples para o escopo do teste, com testes automatizados cobrindo as regras de negócio mais sensíveis (autorização por perfil e por registro). A documentação complementar (README, dicionário de dados, script SQL e roteiro de avaliação) foi escrita para que o projeto possa ser instalado, executado e avaliado por qualquer pessoa sem depender de contexto adicional.